Figure traceability: US 1 — Dimensionality scaling

Figure 1: Empirical Wall-Clock Scaling

	By report we could show…	By code we currently show…
• **Plot**	Line + min–max seed ribbons	Line + markers, no ribbons
• **X-axis**	`n_features`, linear	`n_features`, log
• **Y-axis**	$\log_{10}$(`runtime_s`)	median `runtime_s`, log
• **Lines / layout**	Color = lib×approx; facet rows= `dataset`, cols= `model`	Color = `method`; no facets — click dataset/model/approx filters
Read	Scaling limits + model overhead per dataset×model	One median curve per method, all data pooled
Steps	1. CSV: `n_features`, `runtime_s`, `library`, `approximator`, `dataset`, `model`, `seed` 2. Filter `budget=512`, approximation rows 3. Facet rows= `dataset`, cols= `model` 4. Per panel: group lib×approx × `n_features`, ribbon = min–max over seeds 5. Map x= `n_features` (linear), y=$\log_{10}$(`runtime_s`)	1. Filter `computation_type=approximation` 2. Click filters: dataset, model, approximator, n_features 3. `method = library + approximator` 4. Group `method × n_features` → median over seeds/datasets/models 5. `fig_cost_vs_features(df, "runtime_s")`
Verdict	**Report** — only facets show model overhead	Needs dataset+model filter to match claim

Figure traceability: US 1 — Dimensionality scaling

Figure 2: Model Evaluation Footprint

	By report we could show…	By code we currently show…
• **Plot**	Multi-line grid	Line + markers
• **X-axis**	`n_features`, linear	`n_features`, log
• **Y-axis**	`n_model_evals`, linear	median `n_model_evals`, log
• **Lines / layout**	Same as Fig 1 — lib×approx, faceted by dataset×model	Same chart as Fig 1 — click metric toggle → `n_model_evals`
Read	Isolate sampling cost vs software overhead	Huge eval range readable on log axis, no facet breakdown
Steps	Same as Fig 1, y= `n_model_evals` (linear)	Same as Fig 1 — toggle y to `n_model_evals`; `fig_cost_vs_features(df, "n_model_evals")`
Verdict	**Split** — report facets answer "why fast?"; code log-y fits 6k→237M range	Shares UI with Fig 1

Figure traceability: US 1 — Dimensionality scaling

Figure 3: Relative Attribution Error Decay

	By report we could show…	By code we currently show…
• **Plot**	Line + error bounds	Line, no bounds
• **X-axis**	`n_features`	`n_features`, log
• **Y-axis**	`relative_mae` from `pairwise_metrics`	median `relative_mae`, log
• **Lines / layout**	Color = `library`, aggregated across datasets	Color = `method` (lib×approx); no facets
Read	How each library degrades as M outgrows budget=512	Kernel vs permutation separated
Steps	1. CSV: `n_features`, `relative_mae` vs `shap_true_value` 2. Filter approximation, `budget=512` 3. Group `library × n_features`, error bounds over seeds 4. Map x= `n_features`, y= `relative_mae`	1. Extract `relative_mae` from `pairwise_metrics["shap_true_value"]` 2. Click filters + toggle → `relative_mae` 3. Drop values ≤ 1e-6 4. Group `method × n_features` → median 5. `fig_quality_vs_features(df, "relative_mae")`
Verdict	**Code** for diagnosis; **report** for simpler library story	Shares quality chart with Fig 4

Figure traceability: US 1 — Dimensionality scaling

Figure 4: Feature Importance Ranking Consistency

	By report we could show…	By code we currently show…
• **Plot**	Scatter + LOESS per method	Line of medians
• **X-axis**	`n_features`	`n_features`, log
• **Y-axis**	`mean_sample_rho` ∈ [-1, 1]	median ρ, linear [0, 1.08] + ρ=0.9 line
• **Lines / layout**	Color = lib×approx; one point per run	Color = `method`; seeds aggregated away
Read	Does ranking collapse into noise at high M?	Headline ρ trend per method
Steps	1. CSV: `n_features`, `mean_sample_rho` per run 2. Scatter per run, color=lib×approx 3. LOESS trendline per method 4. Map x=`n_features`, y=`mean_sample_rho`	1. Extract ρ from `pairwise_metrics["shap_true_value"]` 2. Toggle → `mean_sample_rho` 3. Group `method × n_features` → median 4. `fig_quality_vs_features(df, "mean_sample_rho")`
Verdict	**Report** if arguing ranking collapse	**Code** for quick trend only

Figure traceability: US 1 — Dimensionality scaling

Figure 5: Efficiency Axiom Gaps

	By report we could show…	By code we currently show…
• **Plot**	Side-by-side box plots	Not built (line-over-M possible but wrong form)
• **X-axis**	`library`	— would be `n_features` if forced
• **Y-axis**	$\log_{10}$(`relative_additivity_gap`)	— median line only
• **Lines / layout**	Two panels: low-dim M≤16 vs high-dim M=256	No facet, not in UI toggle
Read	Which libraries guarantee additivity structurally vs asymptotically	Cannot answer this claim
Steps	1. CSV: `relative_additivity_gap` , `library` , `n_features` 2. Split panels: M≤16 vs M=256 3. Box plot per panel: x= `library` , y=$\log_{10}$(gap)	1. Column exists in CSV 2. `fig_quality_vs_features("relative_additivity_gap")` exists but not exposed 3. No dedicated figure
Verdict	**Report only** — right question, right form	**Missing**

Figure traceability: US 1 — Dimensionality scaling

Figure 6: Failure Profile / Completion Heatmap

	By report we could show…	By code we currently show…
• **Plot**	2D heatmap	Heatmap
• **X-axis**	`library × approximator`	`n_features`
• **Y-axis**	`dataset × model` (by complexity)	`method`
• **Lines / layout**	Cell = seed completion 3/3 vs timeout/DNF	Cell = mean(`is_failure`) %; click dataset/model filters
Read	Deployment map: can I run this combo on this case?	Dimension map: at which M does method fail?
Steps	1. CSV: `library` , `approximator` , `dataset` , `model` , `seed` , completion 2. x=lib×approx, y=dataset×model 3. Cell = seed completion rate 4. Green=3/3, red=DNF/timeout	1. `is_failure = relative_mae > 1.0 or NaN` 2. `method = library + approximator` 3. Click dataset/model filters 4. Group `method × n_features` → mean(`is_failure`) 5. `fig_failure_heatmap_by_features(df)`
Verdict	**Different questions** — not interchangeable	Answers dimension breakpoints, not deployment limits

Figure traceability: US 2 — Approximation accuracy & budget efficiency

Figure 7: Absolute Error Decay Profiles

	By report we could show…	By code we currently show…
• **Plot**	Grouped bar chart, 100% percentile intervals `errorbar=("pi",100)`	Point + line plot, no intervals
• **X-axis**	budget N ∈ {64, 512}	budget, linear
• **Y-axis**	`relative_mae` vs exact oracle	median `relative_mae`, linear
• **Lines / layout**	Bars = lib×approx; facet cols= `n_background`, rows= `dataset`	Lines = method; n_bg as line style (solid/circle=50, dashed/diamond=200); no facets
Read	Interaction effect: n_bg 200 + budget 64 = error penalty	Same interaction visible via dashed-vs-solid gap; UI warns for n_bg=200 × budget=64
Steps	1. CSV: `budget`, `relative_mae` vs `shap_true_value`, `library`, `approximator`, `n_background`, `dataset` 2. Filter approximation rows 3. Facet cols= `n_background`, rows= `dataset` 4. Bar per lib×approx at each budget, pi-100 interval over seeds/models 5. Map x= `budget`, y= `relative_mae`	1. Extract `relative_mae` from `pairwise_metrics["shap_true_value"]` 2. Exclude failures (`relative_mae > 1.0` or NaN) 3. Click metric toggle → `relative_mae` ; n_bg/budget filters 4. Group `method × budget × n_background` → median over seeds/models/datasets 5. `fig_budget_quality_lines(df, "relative_mae")`
Verdict	**Report** — facets isolate the n_bg interaction; bars + pi-100 show spread	Same effect compactly, but dataset differences and variance hidden

Figure traceability: US 2 — Approximation accuracy & budget efficiency

Figure 8: Feature Importance Ordering Stability

	By report we could show…	By code we currently show…
• **Plot**	Grouped point + line plot	Line + markers per method × n_bg
• **X-axis**	budget N	budget, linear
• **Y-axis**	`mean_sample_rho`, focus band [0.8, 1.0]	median ρ, auto-zoomed to data + ρ=0.9 line
• **Lines / layout**	By method	By method × n_background (style-split)
Read	Is budget 64 enough to trust the ranking, or is 512 needed?	Same question — this is the page's default chart
Steps	1. CSV: `budget`, `mean_sample_rho` vs oracle 2. Group method × budget 3. Map x= `budget`, y=ρ, zoom [0.8, 1.0]	1. Extract ρ from `pairwise_metrics["shap_true_value"]` 2. Exclude failures 3. Group `method × budget × n_background` → median 4. Auto y-zoom [min−0.04, max+0.03]; ρ=0.9 reference 5. `fig_budget_quality_lines(df, "mean_sample_rho")`
Verdict	**Match** — closest report↔code pair in US2	Code extra: `fig_distribution` box+strip shows per-run spread the report omits

Figure traceability: US 2 — Approximation accuracy & budget efficiency

Figure 9: Axiomatic Integrity Check

	By report we could show…	By code we currently show…
• **Plot**	Box-and-whisker plot	Not implemented on the RQ2 page
• **X-axis**	`library`	—
• **Y-axis**	`relative_additivity_gap`, linear	—
• **Lines / layout**	One box per library; approximators near 1e-16; `shap_true_value` anomaly 4–22% at n_bg=200	—
Read	Efficiency axiom holds at machine epsilon; exact backend breaks at large n_bg	Cannot be shown: `load_data()` keeps only approximation rows — the anomalous `true_value` rows are dropped before any chart
Steps	1. CSV: `relative_additivity_gap`, `library`, `n_background`, `backend` 2. Include BOTH approximation and `shap_true_value` rows 3. Box plot x= `library`, y=gap 4. Highlight true_value @ n_bg=200 anomaly	1. Column exists in CSV (range 0 → 0.81; anomaly rows confirmed in raw data) 2. `load_data()` filters `computation_type == "approximation"` → anomaly rows excluded 3. No box-plot builder for this metric; not in any UI toggle
Verdict	**Report only** — and the code pipeline actively hides the headline anomaly	**Missing**; needs loader change + new figure to verify the 4–22% claim

Figure traceability: US 2 — Approximation accuracy & budget efficiency

Figure 10: Practitioner's Pareto Frontier

	By report we could show…	By code we currently show…
• **Plot**	Pareto scatter with explicit frontier	Scatter, no frontier line drawn
• **X-axis**	`runtime_s`	median `runtime_s` (or `n_model_evals` via toggle), linear
• **Y-axis**	accuracy = 1 − `relative_mae`	median ρ (default) or `relative_mae` (log) — never 1−MAE
• **Lines / layout**	Marker = (N × n_background) config	Circle=n_bg 50, diamond=n_bg 200; red border = n_bg 200 × budget ≤ 64; color = library
Read	Upper-left = Pareto-optimal picks for deployment	Same trade-off readable, but optimal set not marked
Steps	1. CSV: `runtime_s`, `relative_mae`, `budget`, `n_background` 2. Compute y = 1 − `relative_mae` 3. One point per lib×approx×budget×n_bg 4. Mark Pareto frontier (min runtime, max accuracy)	1. Extract quality from `pairwise_metrics["shap_true_value"]` 2. Click x/y toggles (y default = ρ) 3. Group `method × budget × n_background` → median over seeds/models/datasets 4. `fig_quality_vs_cost_rq2(df, x, y)` — no `pareto_mark` applied (helper exists in `data.py` but unused here)
Verdict	**Report** for the deliverable — frontier is the point of the figure	Code has all ingredients (`pareto_mark` exists); one wiring step from matching